

AI335L Deep Learning Lab
Experiment No. 10
Deadline: May 10, 2026 12:00 AM

Lab Task (Phase 3)

Core Deep Architecture: CNN, RNN, or Transformer

Course	Deep Learning
Phase	3 of 6 (Week 3)
Submission Type	Updated GitHub repository + Architecture Report (PDF) + Training Logs
Recommended Length (Report)	10–14 pages

1. From Baseline to Backbone

Last week you built a working pipeline and a baseline MLP. This week you replace the MLP with the deep architecture that will actually solve your problem. Whatever you proposed in Phase 1, whether a CNN for images, an LSTM for sequences, or a Transformer for language or long-range dependencies, now becomes real code.

The hardest part of this week is not implementation. PyTorch and TensorFlow have made the mechanics straightforward. The hard part is making good architectural choices and training the model in a way you can defend. A model that hits 92% accuracy through luck is worth less than one at 87% that you can explain in detail. Grading reflects this. Half the rubric is about reasoning, not raw numbers.

2. Learning Objectives

By the end of Phase 3, you will be able to:

- Choose a deep architecture appropriate to your problem and defend the choice with technical reasoning

- Implement a CNN, RNN/LSTM/GRU, or Transformer either from scratch or via well-justified use of pretrained backbones
- Diagnose training pathologies: vanishing/exploding gradients, dead neurons, loss plateaus, overfitting, underfitting
- Apply regularization (Dropout, BatchNorm, LayerNorm, weight decay, gradient clipping) and explain its effect
- Run controlled experiments comparing your deep model against the Phase 2 baseline
- Visualize what your model has learned through filters, attention maps, hidden state activations, or saliency

3. Architecture Tracks

You will follow ONE of these tracks based on your problem domain. Crossing tracks is allowed and encouraged when the project genuinely needs it (for example, a CNN encoder feeding into a Transformer decoder for image captioning).

Track	Use If Your Problem Involves	Required Architectural Elements
CNN	Images, video frames, spectrograms, 2D/3D spatial signals, dense pixel prediction	Convolution + pooling + BatchNorm + at least one form of skip/residual connection if depth > 6 layers
RNN / LSTM / GRU	Time series, sensor streams, short text, speech, anything with strict temporal order	Recurrent cells, gradient clipping, sequence packing/padding handled correctly, bidirectional where it makes sense
Transformer	Language, long sequences, multimodal inputs, problems with non-local dependencies	Multi-head self-attention, positional encoding, LayerNorm, learning-rate warmup, attention masking where required

Note on pretrained models: you may use a pretrained backbone (ResNet, ViT, BERT, GPT-2, etc.) as the core of your architecture, but only if you can articulate which layers you froze, which you fine-tuned, and why. “We used BERT” is not an architecture choice. “We used BERT-base, froze the embedding layer and bottom 6 encoder layers, and fine-tuned the top 6 plus a classification head with a learning rate of 2e-5” is.

4. What You Will Submit

Four artifacts, all linked from your team’s GitHub repository:

Deliverable	Description	Format
Updated Repository	Model code under <code>src/models/</code> , updated training loop, architecture diagram in <code>reports/</code> .	GitHub URL + tag
Trained Checkpoints	Best checkpoint by validation metric, plus the corresponding config that produced it.	.pt / .h5 + config.yaml
Training Logs	Full training curves from your tracker (TensorBoard, W&B, MLflow). All runs from this week, not just the best one.	Public link or exported screenshots
Architecture Report	Written report covering choice, implementation, training, comparison with baseline, and interpretation.	PDF, 10–14 pages

5. Detailed Task Breakdown

Six tasks, in order. Tasks 5 and 6 depend on a working trained model from Tasks 1–4, so plan accordingly.

Task 1: Architecture Choice & Justification

Before you write a single line of model code, write a one-page architecture justification. This document goes into your report and forces you to think before coding.

It must answer:

- Which track did you choose and why is it the right tool for this problem?
- What specific architecture variant will you use? (e.g., not just “CNN” but “ResNet-18 with the final FC layer replaced by a 3-layer classifier”)
- Are you training from scratch or using a pretrained backbone? If pretrained, which one and why?
- What is the expected input shape, output shape, and approximate parameter count?
- What inductive biases of this architecture match your data? (e.g., translation invariance for CNNs, sequential dependency for RNNs, long-range attention for Transformers)

Task 2: Model Implementation

Implement the architecture as a clean, modular module under `src/models/`. The model class should accept all hyperparameters via its constructor, never read globals or hardcode dimensions.

Required:

- A model class with type-hinted `__init__` accepting all hyperparameters (depth, width, dropout, num_classes, etc.)
- A `forward()` method with shape comments at every major step
- A `summary()` or `print(model)` call at the start of training that lists all layers and parameter counts
- A diagram of your model in `reports/architecture_diagram.png` (Netron, draw.io, or hand-drawn is acceptable)
- A unit test in `tests/` that constructs the model with a tiny config, passes a dummy input through it, and asserts the output shape

Implementation rule: if you copy code from a tutorial or repo, you cite it in a comment AND in the report. Cited code is fine. Uncited copying is plagiarism and is graded as such.

Task 3: Training Infrastructure

Your Phase 2 training loop probably needs upgrades. At minimum, add:

- A learning rate scheduler (StepLR, CosineAnnealingLR, ReduceLROnPlateau, or warmup+decay for Transformers)
- Gradient clipping (essential for RNNs, useful elsewhere)
- Mixed-precision training (`torch.cuda.amp` or `tf.keras.mixed_precision`) if you have a GPU. This roughly halves memory and is free speed.
- A config-driven training script: `python train.py --config configs/cnn_v1.yaml`. No more editing the script to change hyperparameters.
- Resumable training: save optimizer state, scheduler state, and epoch number alongside model weights so a crash does not cost you 10 hours
- Per-epoch logging of train loss, val loss, train metric, val metric, learning rate, and gradient norm

If your dataset is large enough that an epoch takes more than 30 minutes on your hardware, also add a small “debug subset” mode that trains on 1–2% of the data so you can verify the pipeline works end-to-end before launching long runs.

Task 4: Initial Regularization

Train the model in three configurations and log all three. This is your first ablation.

Configuration	What to Run
Unregularized	No dropout, no weight decay, no augmentation. Train until clear overfitting. This shows the model has the capacity to memorize.
Regularized	Add dropout (0.1–0.5 depending on architecture), weight decay (typical $1e-4$ to $1e-2$), and the augmentation pipeline from Phase 2.

Configuration	What to Run
Normalized	Add BatchNorm (CNN), LayerNorm (Transformer), or appropriate normalization for your architecture, on top of regularization.

Plot all three runs on the same axes (one chart for train loss, one for val loss, one for val metric). Discuss what changed and why in the report.

Task 5: Comparison with Baseline

This is the rigour test. Your deep model must be compared to the Phase 2 MLP under conditions that make the comparison fair.

Required comparisons:

- Same train/val/test split
- Same preprocessing pipeline
- Same evaluation metrics, reported with at least 3 random seeds and mean $\pm$ standard deviation
- Approximate parameter count and training time for each model
- A statistical sanity check: if your deep model gain is within one standard deviation of the baseline, do not claim victory

Honesty clause: if the deep model does not beat the baseline yet, say so in the report. This happens, especially with small datasets, and it is more important to diagnose why than to hide it. Phases 4 and 5 give you opportunities to fix it.

Task 6: Interpretation & Visualization

End the week by looking inside the model. Choose at least one visualization appropriate to your architecture:

- **CNN:** first-layer filter visualization, feature maps from intermediate layers on a sample input, Grad-CAM or saliency maps for predictions
- **RNN/LSTM:** hidden-state trajectories over a sequence, gate activations (input/forget/output) for a few example sequences, t-SNE of final hidden states colored by class
- **Transformer:** attention heatmaps for a few attention heads on representative inputs, layer-wise attention entropy, embedding visualization with PCA or t-SNE

Save the visualizations to reports/figures/ and embed them in the report. Each visualization needs a paragraph explaining what it shows and what you concluded from it.

6. Track-Specific Requirements

On top of the common tasks, each track has additional requirements that catch the most common track-specific mistakes.

CNN Track

- Document the receptive field at the final feature map layer. If it is smaller than your input, your model literally cannot see the whole image at once.
- Use BatchNorm if your network is deeper than 6 layers, or justify why you didn't
- Use residual or skip connections for any network deeper than 10 layers
- Augmentation is not optional. Train and validate without augmentation first to see the gap, then add it.
- Document input normalization (mean and standard deviation per channel)

RNN / LSTM / GRU Track

- Use packed sequences (`torch.nn.utils.rnn.pack_padded_sequence`) for variable-length input. Padding without packing wastes compute and biases the loss.
- Gradient clipping is mandatory. Set `clip_grad_norm_` to between 0.5 and 5.0.
- If you use vanilla RNN, you must also report results with LSTM or GRU and discuss the difference. Vanilla RNN alone is rejected unless you have a strong reason.
- Document hidden-state initialization (zeros, learned, or other).
- If your task has bidirectional context available (e.g., classification rather than next-token prediction), use a bidirectional variant.

Transformer Track

- Positional encoding is required. Sinusoidal, learned, RoPE, or ALiBi are all acceptable; document which you used and why.
- Use learning-rate warmup. Transformers without warmup commonly diverge or train very poorly.
- Document the attention mask: causal for autoregressive tasks, padding mask for variable-length input, both for decoders.
- Document Pre-LN vs Post-LN choice. Pre-LN trains more stably; Post-LN matches the original paper. Either is fine if justified.
- If using a pretrained model from HuggingFace, pin the model version (e.g., `bert-base-uncased@<commit>`) and the transformers library version. Models change.

7. Architecture Report Structure

The report (10–14 pages, PDF) must contain:

1. Cover Page & Repository Link

Team, course, GitHub URL, commit hash and tag of submission, date.

2. Architecture Justification

The one-page document from Task 1, expanded with any decisions you revised during the week.

3. Model Architecture

Diagram, layer-by-layer table (layer type, output shape, parameter count), total parameter count, FLOPs estimate (optional but encouraged).

4. Training Setup

Hyperparameters table, optimizer, scheduler, hardware used, total training time, random seeds used.

5. Regularization Ablation

The three-config experiment from Task 4. Loss curves on shared axes, final metrics in a table, written interpretation.

6. Comparison with Baseline

Side-by-side metrics with mean $\pm$ std across seeds. Discussion of where the deep model wins, ties, or loses.

7. Interpretation & Visualization

Filters, attention, or hidden-state visualizations from Task 6 with written analysis.

8. Failure Analysis

Pick 5–10 examples your model gets wrong. What patterns emerge? This sets up Phase 5.

9. Issues Encountered & Plan for Phase 4

What broke this week? What does Phase 4 (transfer learning, generative component, hyperparameter tuning) need to address?

8. Submission Guidelines

- Tag your submission commit: `git tag phase3-submission && git push --tags`
- Submit on the LMS: (a) PDF of architecture report, (b) link to GitHub repo, (c) commit hash, (d) link to your training-tracker dashboard or exported logs
- Best checkpoint must be downloadable. If too large for git, upload to Google Drive or HuggingFace Hub and link from the README.
- Late penalty: 10% per day, no submissions after 72 hours late

9. Evaluation Rubric

Criterion	Weight
Architecture choice & justification	15%

Criterion	Weight
Implementation correctness & code quality	20%
Training infrastructure (configs, scheduler, logging, checkpoints)	10%
Regularization ablation (rigour and analysis)	15%
Baseline comparison (fairness and statistical honesty)	15%
Interpretation & visualization	10%
Architecture report quality	10%
Track-specific requirements	5%

Notice that **only 15% of the grade is the baseline-comparison number itself**. Reasoning, rigour, and analysis carry the rest. A team that beats the baseline through luck without understanding why will score lower than a team that loses to the baseline but diagnoses it correctly.

10. Common Pitfalls

- **Architecture too big for the dataset.** A 50M-parameter Transformer on 5,000 training examples will overfit instantly. Match capacity to data size, then scale up only with evidence.
- **No learning-rate scheduler.** Constant learning rates work for small problems and almost nothing else. Add a scheduler from day one.
- **Forgetting to set the model in eval mode for validation.** `model.eval()` in PyTorch, `training=False` in Keras. Otherwise BatchNorm and Dropout behave wrongly during evaluation and your val numbers lie.
- **Comparing the deep model to the baseline on a different split.** Even subtly different splits invalidate the comparison. Use the exact same indices.
- **One-shot results.** Single-seed numbers are folklore, not science. Three seeds minimum for the comparison table.
- **Tweaking on the test set.** Same rule as Phase 2. The test set stays untouched until Phase 5.
- **Ignoring gradient norm.** Log it. A gradient norm that explodes or collapses to zero tells you exactly what is wrong before the loss curve does.
- **Pretrained model misuse.** Loading bert-base and training the whole thing from scratch wastes the pretrained weights. Either freeze most layers or use a much smaller learning rate on the backbone than on the new head.